

Mesure de la puissance moyenne d'un signal RF

Table des matières

Introduction	3
Prérequis	3
Matériel	3
Logiciels	3
Vérifications préalables	4
1. Principes généraux de mesure	4
1.1 Paramètres	4
1.2 Formulation théorique	4
2. Chaîne de réception et calibration	5
2.1 Principe de la calibration	5
2.2 Modèle de puissance	5
2.3 Communication avec la sonde de puissance (Linux)	6
Vérifications Linux (USB/permissions)	6
Intégration avec les scripts du projet	6
Notes	
3. Architecture logicielle du driver Python	6
Paramètres clés	8
4. Résultat mesures alpha	9
4.1 USRP N210	

Abstract

This document presents a method for power calibration of a Software Defined Radio (SDR) using an additive correction factor expressed in decibels. The objective is to compensate for the discrepancies between the power estimated by the GNU Radio processing chain and the actual RF power at the receiver input. The proposed method relies on a comparative measurement between the SDR and a Rohde & Schwarz NRP33S reference power sensor connected through a splitter. The SDR acquires the IQ samples and estimates the average received power, while the reference power sensor provides a metrological measurement of the RF signal. A set of Python scripts automates the calibration process by iteratively adjusting the calibration factor α until the difference between the two measurements is below a user-defined tolerance threshold. The resulting calibration factor can then be used to improve the accuracy of power measurements performed by the SDR. The procedure can also be repeated at different frequencies in order to characterize the frequency dependence of the reception chain and generate a calibration table $\alpha(f)$ for the SDR under test.

Introduction

Ce document décrit une méthode permettant de calibrer en puissance une carte SDR (*Software Defined Radio*) à l'aide d'un facteur de correction additif exprimé en décibels. L'objectif est de compenser les écarts pouvant exister entre la puissance calculée par la chaîne de traitement GNU Radio et la puissance réellement présente à l'entrée du récepteur.

Le principe consiste à générer un signal RF à l'aide d'un flowgraph GNU Radio, puis à mesurer ce signal simultanément par deux moyens différents :

- une carte SDR associée à un flowgraph GNU Radio réalisant l'estimation de la puissance reçue ;
- une sonde de puissance de référence Rohde & Schwarz NRP33S fournissant une mesure métrologique de la puissance du signal.

La puissance calculée par GNU Radio, notée PGRC,dBm, est comparée à la puissance mesurée par la sonde, notée Ppm,dBm. Un script Python ajuste automatiquement un coefficient de calibration α jusqu'à ce que l'écart entre les deux mesures devienne inférieur à un seuil défini par l'utilisateur. Le coefficient obtenu permet alors de corriger les mesures réalisées par la carte SDR et d'améliorer leur exactitude.

Cette procédure peut être répétée pour plusieurs fréquences afin de caractériser la dépendance fréquentielle de la chaîne de réception et d'établir une table de calibration $\alpha(f)$ propre à la carte SDR utilisée.

Prérequis

Avant d'utiliser les scripts de calibration, les éléments suivants doivent être installés et opérationnels :

Matériel

- Une carte SDR compatible avec GNU Radio (USRP dans le cadre de cette procédure).
- Une sonde de puissance Rohde & Schwarz NRP33S.
- Le câblage RF nécessaire à la connexion du générateur de signal, de la sonde de puissance et de la carte SDR.
- Un ordinateur sous Linux.

Logiciels

- Python 3.
- GNU Radio.
- Les pilotes UHD nécessaires à l'utilisation de l'USRP.
- La bibliothèque Python RSInstrument permettant la communication avec la sonde Rohde & Schwarz via USB ou LAN.
- Les scripts de calibration :
 - compute_alpha.py

- alpha_dump_usrp.py
- pwr_rs_nrp33s_linux.py
- Le flowgraph GNU Radio de mesure de puissance correspondant à la carte SDR utilisée.

Vérifications préalables

Avant de lancer la calibration, il est recommandé de vérifier :

1. que la sonde NRP33S est correctement détectée par le système ;
2. que la bibliothèque RSInstrument permet d'établir une communication avec la sonde ;
3. que l'USRP est visible via les outils UHD ;
4. que le flowgraph GNU Radio de mesure fonctionne correctement ;
5. que les scripts Python peuvent être exécutés sans erreur et disposent des droits d'accès nécessaires aux périphériques.

Une fois ces prérequis validés, la procédure de calibration peut être exécutée afin de déterminer le coefficient de correction associé à la carte SDR considérée.

1. Principes généraux de mesure

La puissance moyenne est estimée à partir des échantillons IQ du signal numérisé sur une fenêtre temporelle définie.

1.1 Paramètres

- T_{int} : temps d'intégration (s)
- F_s : fréquence d'échantillonnage (Hz)
- $N = T_{\text{int}} * F_s$: nombre d'échantillons
- R : impédance de référence (50 Ω)
- α : facteur de calibration

1.2 Formulation théorique

La puissance moyenne numérique est calculée à partir des échantillons IQ selon :

$$P = \frac{1}{NR} \sum_{n=1}^N |x[n]|^2$$

Un facteur de calibration α est nécessaire pour convertir les unités numériques (échantillons codés sur 2^n bits de x) en puissance physique (tension en Volt). Nous détaillerons dans la partie suivante son influence sur l'expression de la puissance moyenne numérique.

2. Chaîne de réception et calibration

Cette section décrit la chaîne RF complète et la nécessité d'une calibration pour intégrer les gains et pertes internes non accessibles.

2.1 Principe de la calibration

Une mesure comparative est réalisée entre un power meter de référence et la mesure numérique issue du SDR via un montage utilisant un splitter.

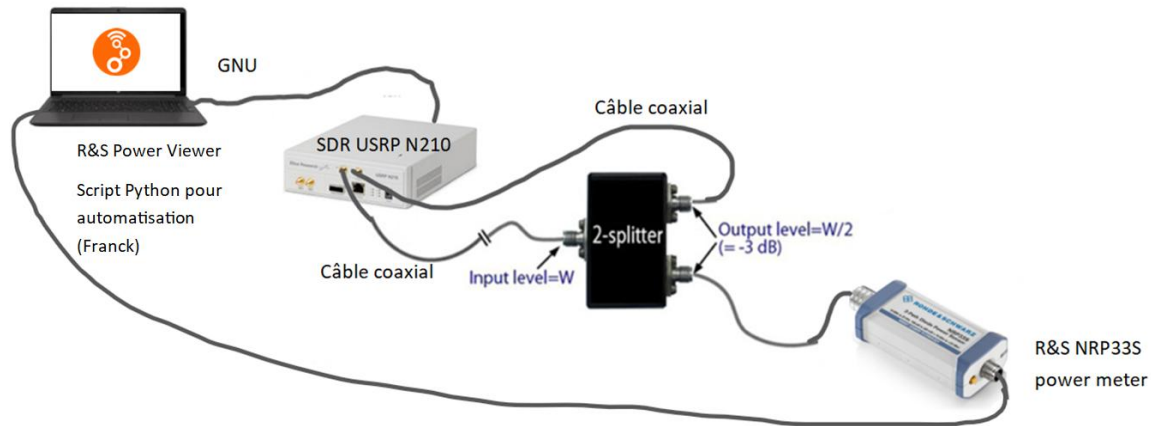


Figure 1 : Schématisation du banc d'essai pour la détermination de α

2.2 Modèle de puissance

La puissance numérique mesurée peut s'écrire :

$$P_{SDR_dBm} = 10 \cdot \log \left(\frac{1}{NR} \sum_{n=1}^N |x[n]|^2 \right) - G_{Rx}$$

Avec G_{Rx} le gain du signal dans la chaîne de réception du SDR. Cette expression va nous permettre de définir la facteur de calibration (α).

$$P_{Power_meter_dBm} = P_{SDR_dBm} + \alpha$$

Voici comment cette expression est retranscrite sur GRC dans le cas où le signal est converti analogique par le SDR USRP Source.

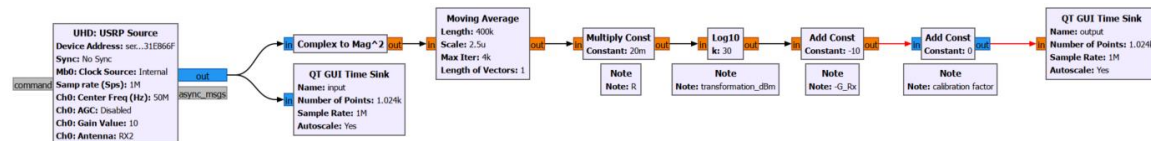


Figure 2 : Flowgraph GRC de la chaîne de mesure de la puissance du signal numérique

Il faut d'ailleurs que l'USRP en Rx et en Tx soient dans le même graph GnuRadio pour le faire fonctionner en full-duplex.

2.3 Communication avec la sonde de puissance (Linux)

Objectif — permettre à Linux et à Python de dialoguer avec la sonde de puissance (R&S NRPxx), puis d'exposer la puissance de référence P_{pm} (dBm) aux scripts d'automatisation.

Vérifications Linux (USB/permissions)

1) La sonde est-elle vue ?

```
$ lsusb
```

2) Les devices sont-ils accessibles en écriture ?

```
$ ls -l /dev/bus/usb/**/**
```

→ viser des droits de type crw-rw-rw- (0666) pour l'utilisateur.

Si bug (autrement vérification non nécessaire) :

Règles udev

- Identifier idVendor et idProduct avec lsusb.
- Règle type : `SUBSYSTEM=="usb", ATTR{idVendor}=="<VID>", MODE="0666"`
- Recharger :


```
$ sudo udevadm control --reload-rules
```

```
$ sudo udevadm trigger
```
- Débrancher/rebrancher la sonde.

Intégration avec les scripts du projet

Utiliser le script dédié (ex. `pwr_rs_nrp33s_linux.py`) pour retourner P_{pm_dBm} . Les scripts d'automatisation attendent une ligne standardisée : '`P_pm_dBm=<valeur>`' imprimée sur la sortie standard.

Note : Vérifier que la fréquence de mesure configurée sur la sonde est bien dans sa plage nominale pour éviter les casses.

3. Architecture logicielle du driver Python

Cette partie présente l'architecture logicielle mise en œuvre pour déterminer automatiquement le facteur de calibration associé à une carte SDR donnée. L'objectif est

d'estimer un coefficient de correction α permettant de compenser les écarts entre la puissance mesurée par un instrument de référence et celle calculée par la chaîne de réception SDR.

L'architecture repose sur une boucle de calibration automatisée pilotée par un ensemble de scripts Python interagissant avec un flowgraph GNU Radio et un powermeter de référence. Le script principal, **compute_alpha.py**, joue le rôle d'orchestrateur. Il exécute successivement les mesures, récupère les résultats produits par les différents instruments et ajuste itérativement la valeur du facteur de calibration.

La mesure de puissance côté SDR est réalisée par le script **alpha_dump_usrp.py**, qui instancie et exécute le flowgraph GNU Radio correspondant à la carte utilisée. Nous pourrions penser plus tard à produire d'autres scripts largement similaires qui soient utilisables pour la calibration d'autres familles de carte SDR. Leurs noms sont déjà référencés dans le script **compute_alpha.py**.

- USRP_B200_Mini > alpha_dump_usrp.py,
- USRP_N210 > alpha_dump_usrp.py,
- USRP_E310 > alpha_dump_usrp.py,
- SDR_ADALM_PLUTO > alpha_dump_pluto.py,
- Z706 > alpha_dump_fmcomms.py,
- ZBOARD > alpha_dump_fmcomms.py,
- PICO_SDR > alpha_dump_fmcomms.py.

Ce flowgraph effectue l'acquisition des échantillons IQ, le calcul de la puissance moyenne reçue ainsi que l'application du facteur de correction α . Une sonde interne (*probe*) permet ensuite de récupérer la valeur de puissance calculée, notée PGRC,dBm.

Parallèlement, le script **pwr_rs_nrp33s_linux.py** pilote le powermeter Rohde & Schwarz NRP33S utilisé comme référence métrologique. La puissance mesurée par cet instrument est notée Ppm,dBm.

À chaque itération, **compute_alpha.py** compare les deux mesures. L'écart de puissance est calculé selon :

$$\Delta P = \text{PGRC,dBm} - \text{Ppm,dBm}$$

Lorsque cet écart est supérieur au seuil de tolérance fixé, la valeur de α est modifiée puis une nouvelle mesure est effectuée. Le processus est répété jusqu'à obtenir :

$$|\text{PGRC,dBm} - \text{Ppm,dBm}| \leq \text{threshold}$$

où *threshold* représente le critère de convergence exprimé en décibels.

Cette architecture modulaire présente plusieurs avantages. Elle permet de dissocier la mesure SDR, la mesure de référence et l'algorithme de calibration, facilitant ainsi la maintenance du logiciel et son adaptation à différents modèles de cartes SDR ou d'instruments de mesure. Elle permet également d'automatiser entièrement la génération des coefficients de calibration nécessaires à l'établissement des courbes $\alpha(f)$ utilisées lors de la caractérisation fréquentielle des équipements.

La Figure 3 présente l'architecture fonctionnelle de l'outil Python et les échanges entre les différents modules logiciels.

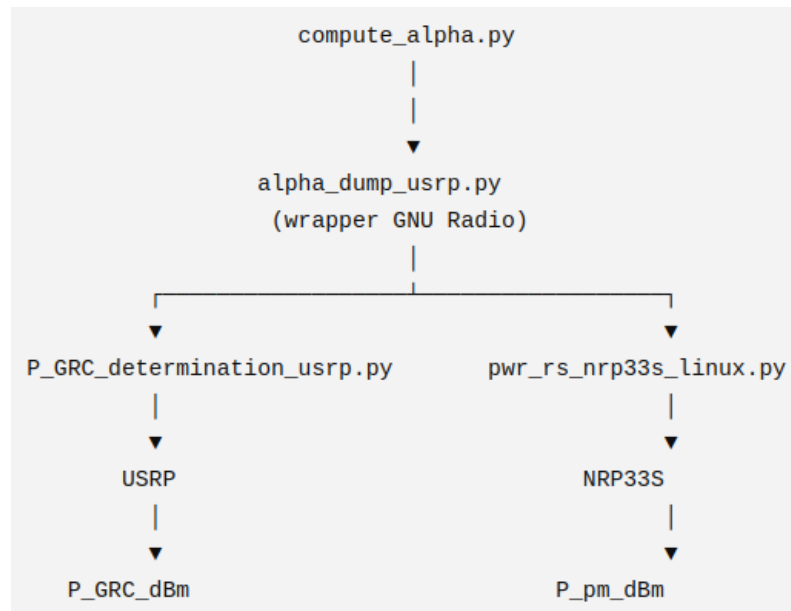


Figure 3 : Architecture fonctionnelle de l'outil Python

Paramètres clés

threshold (dB) > critère d'arrêt des itérations pour l'actualisation de la valeur de alpha ;

max_iter > garde-fou évitant que le code ne tourne trop longtemps ce qui pourrait engendrer de la dégradation matérielle ;

max_step > pour éviter des bonds en dB du paramètre de calibration trop important,

pm_args — commande pour la sonde.

L'usage de ce script nous permettra de produire un tableau $\alpha(f)$ pour un SDR donné et de tracer des courbes de calibration α en fonction de la fréquence (représentation graphique du tableau précédent).

4. Résultat mesures alpha

Not in this version.